

Energy usage prediction

Abstract

Accurate building energy consumption forecasting plays an important role in improving energy efficiency, reducing operational costs, and supporting sustainable smart-building management systems. This project focuses on predicting hourly building energy consumption using the ASHRAE Great Energy Predictor III dataset, which contains large-scale building metadata, weather data, and meter readings from multiple sites and building types. Extensive preprocessing and feature engineering techniques were applied, including missing-value imputation, anomaly handling, logarithmic transformations, lag features, rolling weather statistics, and temporal feature extraction. Multiple machine learning models, including Linear Regression, Random Forest, XGBoost, LightGBM and CatBoost, were trained and evaluated using Root Mean Squared Logarithmic Error (RMSLE). Experimental results showed that gradient-boosting models consistently outperformed traditional regression approaches for this structured tabular dataset. Among all tested models, CatBoost achieved the best performance with an RMSLE of approximately 0.5022 after hyperparameter tuning. And soft voting ensemble method using LightGBM and CatBoost achieved the best overall result of 0.4997, demonstrating the effectiveness of ensemble-based boosting techniques for large-scale energy consumption forecasting.

1 Introduction

Energy efficiency claims are usually settled by comparison rather than by direct measurement. The saving attributed to a retrofit is the difference between observed consumption and the consumption a baseline model would have predicted under the same weather, calendar and operating conditions [1]. The ASHRAE Great Energy Predictor III data set, released for the 2019 Kaggle competition, was assembled with that kind of counterfactual estimation in mind. It pairs hourly meter readings for electricity, chilled water, steam and hot water with static building metadata and with contemporaneous weather observations recorded at sixteen sites.

Modelling these meters is harder than the size of the data alone suggests. Demand depends on building use, gross floor area, hour of day, day of week, season, and recent consumption history, and the interactions between those factors are non-linear. Outdoor temperature has a piecewise effect: a one-degree shift inside the comfort band moves load very little, but the same shift in a heating- or cooling-dominated period changes load substantially. The data themselves carry several documented artefacts. Roughly 60% of `year_built` and 83% of `floor_count` are missing in the metadata. The weather table has station-specific gaps in cloud cover, precipitation, sea-level pressure and wind direction. Site 0 records implausibly low electricity readings before 20 May 2016, and a non-trivial fraction of building-meter pairs show long zero-reading runs that look more like outages than genuine zero load.

The target variable is heavily right-skewed. All models were fitted on $\log 1p(\text{meter_reading})$ and assessed with RMSLE, since proportional rather than absolute error is the only criterion that places a small office and a large hospital on a comparable footing when consumption volumes span several orders of magnitude.

What follows develops three threads in turn. The ingestion stage joins `train`, `building_metadata` and `weather_train` into a single 20,216,100-row table, with every one of the 1,449 building identifiers preserved. The cleaning stage translates the Sprint 1 audit findings into explicit rules on the merged table: invalid meter records are removed, metadata is imputed at the primary-use level, weather gaps are closed station-by-station before the join, and a coherent set of calendar, lag, rolling, weather-derivative and building-derivative features is engineered. The modelling stage then compares five families (XGBoost, LightGBM, CatBoost, Random Forest and Linear Regression) under a strictly chronological split. January to October 2016 is used for training and November to December 2016 is

held out for validation, so that the short-range temporal autocorrelation of meter readings cannot be exploited by a random split.

2 Literature Review

Recent building energy prediction literature shows that traditional machine learning, especially tree-based ensemble models, is more common than standalone deep learning for structured tabular datasets. The ASHRAE Great Energy Predictor III overview paper reports a large real-world dataset with 1,448 buildings, 2,380 meters and over 20 million training records, evaluated using RMSLE [1]. The top Kaggle solutions mainly relied on ensemble gradient-boosting methods such as LightGBM, XGBoost, and CatBoost, sometimes combined with neural networks [2]. This shows that deep learning appeared in some ensembles, but boosting models were central.

2.1 Insights from the ASHRAE Great Energy Predictor III Competition

A detailed analysis of the top-performing teams in the ASHRAE Great Energy Predictor III competition further validates the effectiveness of gradient-boosting approaches and extensive preprocessing for large-scale building energy prediction. The first-place team, Motoki and Yamashita, found that data cleaning and fixing anomalies were the most important factors for model performance. Their solution included removing unusual meter readings, correcting inconsistencies in Site 0 units, filling in missing weather data, adjusting timestamps to local time, and applying a log1p transformation to the target variable. Their final model combined CatBoost, LightGBM, and MLP models trained on various levels of detail using extensive feature engineering. This included lag features, cyclical time encoding, target encoding, and weather smoothing techniques [3]. The second-place team, cHa0s, also focused on LightGBM and XGBoost models. They emphasized modeling each site separately and highlighted the importance of lag-based features, like weekly lag (`lag_168`), and rolling weather statistics [4]. Similarly, the third-place team, eagle4, showed that averaging several well-validated LightGBM models yielded better results than depending solely on complex neural network structures [5]. Across the top solutions in the competition, common strategies included thorough data cleaning, log-transformed targets, time-aware validation methods, and ensemble learning with gradient-boosting models.

2.2 Effectiveness of Gradient-Boosting Methods

Several papers confirm the strength of boosting methods. Miller et al. found that gradient boosting machines, especially LightGBM, performed best in the ASHRAE competition when combined with preprocessing, imputation and outlier removal [6]. A comparison of LightGBM, XGBoost and CatBoost also showed that boosting models capture nonlinear relationships between weather, building characteristics and energy use effectively [7]. Similarly, an XGBoost quantile regression study compared XGBoost with ANN, SVM and Random Forest, showing that boosting can provide strong accuracy and interpretability through feature-importance methods such as SHAP [8].

CatBoost-specific studies further support its effectiveness for tabular energy prediction tasks. A CatBoost model tuned using random search and Bayesian optimisation achieved very high accuracy for building energy prediction and outperformed several baseline models [9]. Another CatBoost study using meta-heuristic optimisation also showed strong performance on feature-based energy data, although optimisation increased computational cost [10]. Overall, gradient-boosting models such as LightGBM, XGBoost and CatBoost are widely adopted due to their ability to model nonlinear feature interactions and deliver strong performance on structured building energy data.

3 Methodology

This project follows an iterative machine learning pipeline inspired by the approaches used in the top-performing solutions of the ASHRAE Great Energy Predictor III competition. The overall workflow of the system is illustrated in figure 1. The methodology begins with integrating the building metadata, weather, and meter-reading datasets, followed by preprocessing, feature engineering, model training, and evaluation.

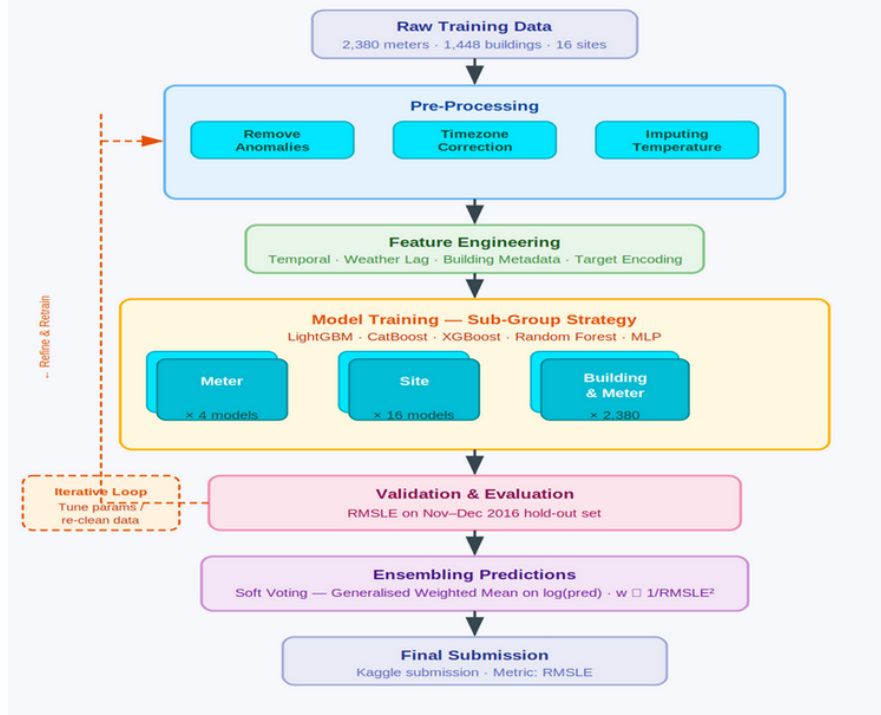


Figure 1: Iterative Model Pipeline

3.1 Data Preprocessing and Analysis

3.1.1 Building Metadata Dataset

The building metadata dataset contains descriptive information about each building included in the ASHRAE energy prediction dataset. It includes attributes such as building ID, site ID, primary building usage category, square footage, year built, and floor count. It contains data for 1449 buildings. This dataset plays an important role in providing contextual information that helps machine learning models understand how building characteristics influence energy consumption patterns.

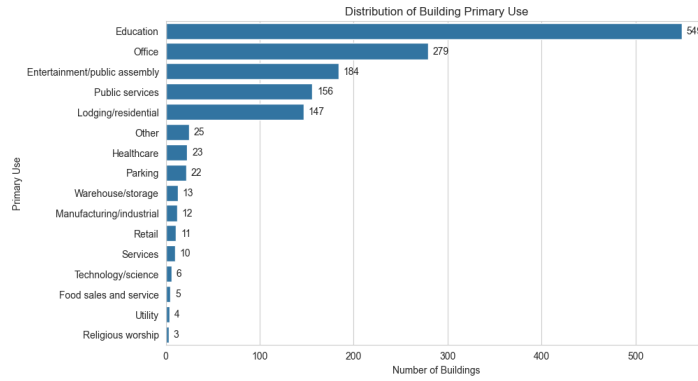


Figure 2: Primary Use Count Distribution

The dataset represents a diverse collection of buildings distributed across multiple geographic sites[2]. The `site_id` attribute indicates different weather locations, allowing the integration of local climate information with building-level energy usage. Buildings in the dataset belong to 16 different primary usage categories, including education, office, healthcare, lodging/residential, retail, technology/science, and warehouse/storage. Education had the highest number of buildings, with 549 entries, followed by office buildings with 279 entries. In contrast, fewer than 10 buildings belonged to categories such as Technology/Science, Food Sales and Service, Utility, and Religious Worship. Since

these categories are nominal and non-ordinal, One-Hot Encoding was applied to convert them into machine-readable binary features without introducing artificial ordering relationships.

The `square_feet` feature showed a highly skewed distribution with a very large size range between small and large buildings[3,4]. To reduce skewness and stabilize variance, a logarithmic transformation (`log_square_feet`) was applied using the `log1p()` function. The `year_built` feature was used to derive a new attribute called `building_age`, calculated relative to the dataset year (2016). This transformation allowed the model to capture the effect of building age more effectively than using raw construction years.

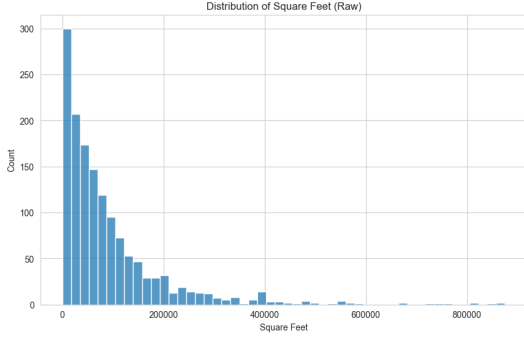


Figure 3: Raw Square Feet Distribution

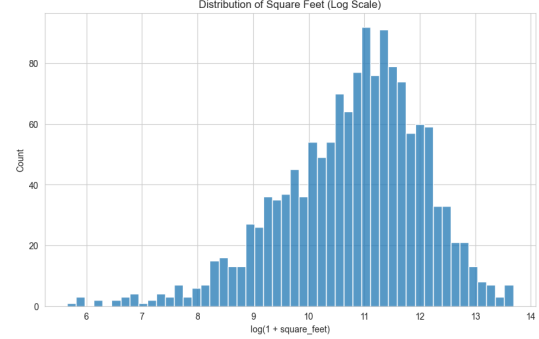


Figure 4: Log1p Transformed Square Feet

Several missing-data handling decisions were made during preprocessing. The `floor_count` attribute contained a large proportion of missing values; however, instead of removing the feature entirely, a new binary feature called `missing_floor_count_flag` was created to preserve information about missingness. Missing floor counts were then imputed using grouped median values based on `primary_use`, `site_id`, and building-size categories derived from square footage. Missing `year_built` values were similarly imputed using grouped medians by building type and site. These decisions were made to retain potentially useful structural information while minimizing information loss and maintaining dataset consistency for model training.

3.1.2 Weather Dataset

Weather data were cleaned before merging because each station-hour reading is shared by all buildings at the same site. Imputing after the join would duplicate one missing station reading across many building rows and distort fill statistics. Therefore, `weather_train.csv` and `weather_test.csv` were sorted by `site_id` and `timestamp`, processed using the same station-level preprocessing function, and then joined to the meter data. The training merge preserved all 20,216,100 rows, confirming that missingness originated from the weather variables rather than from the join itself.

The main missing-value gaps were found in `cloud_coverage` (approximately 44%), `precip_depth_1_hr` (approximately 19%), `wind_direction` (approximately 7%), and `sea_level_pressure` (approximately 6%). In contrast, temperature and wind-speed variables were mostly complete. Gradual variables such as `air_temperature` and `dew_temperature` were linearly interpolated within each site. The `cloud_coverage` feature was forward-filled and backward-filled before applying site-level and global median fallback values. `precip_depth_1_hr` was filled with zero and paired with a missingness indicator feature. `sea_level_pressure`, `wind_direction`, and remaining weather nulls were handled using local rolling medians, site-level medians, and final forward/backward filling.

Two additional weather features were engineered after imputation. The first was `relative_humidity`, computed from air temperature and dew temperature and clipped to the valid range of 0–100%. The second was `temp_diff_from_comfort`, defined as the absolute difference from a 21°C reference temperature. Processed floating-point weather columns were cast to `float32` before the large train and test merges to reduce memory usage without affecting the precision of tree-based regression models.

After combining the cleaned weather streams with the building-meter data, the final training table contained 17,489,805 usable rows. A chronological split was applied using January to October 2016 for training and November to December 2016 for validation. This avoided placing adjacent readings

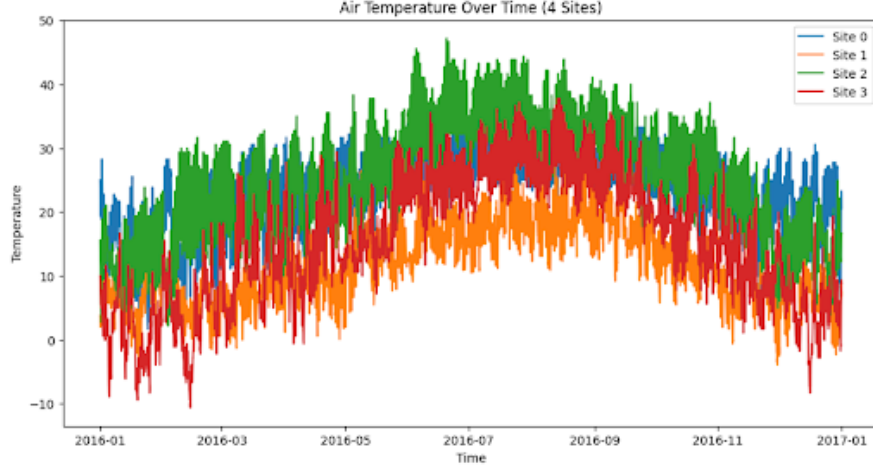


Figure 5: Air Temperature by Timestamp for the First Four Sites in `weather_train.csv`

from the same building-meter pair into both datasets and more accurately reflected the future-period structure of the hidden test set.

3.1.3 Train Dataset

The training dataset contains over 20 million hourly energy readings from 1,441 buildings across four meter types: electricity (0), chilled water (1), steam (2), and hot water (3), covering the year 2016. The dataset includes four primary columns: `building_id`, `meter`, `timestamp`, and `meter_reading`. No missing values were observed in these columns; however, exploratory analysis revealed data quality issues such as skewed distributions and anomalous zero readings that required further processing.

Energy consumption exhibits clear daily patterns, with peak usage typically occurring during working hours (approximately 12 PM to 4 PM) and lower consumption during night hours. This reflects building occupancy and operational schedules. These patterns justify the inclusion of time-based features such as hour and business-hour indicators. Energy usage varies significantly across seasons as well. Chilled water demand increases in warmer periods due to cooling requirements, while steam usage is higher in colder periods due to heating demand. This highlights the importance of seasonal features such as month and season in capturing energy consumption behaviour.

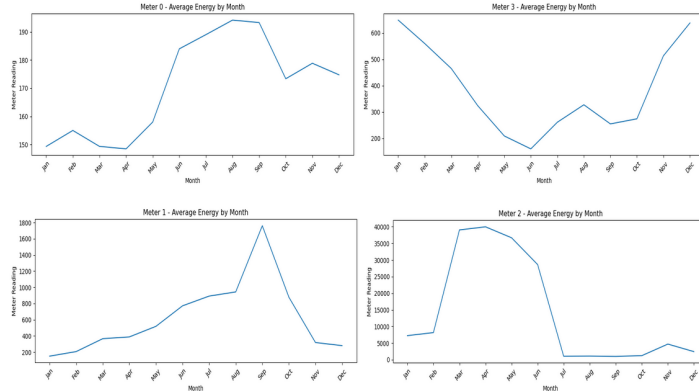


Figure 6: Average Energy by Month for each meter type

Energy consumption is consistently higher on weekdays compared to weekends, reflecting reduced building activity during non-working days. This supports the inclusion of a weekend indicator feature to capture operational differences.

Feature engineering focused on capturing temporal patterns and dependencies in energy consumption. Time-based features such as hour, day_of_week, month, and season were extracted from times-

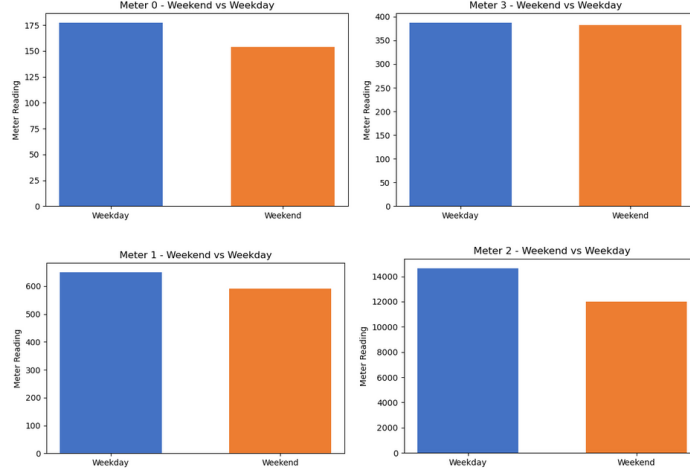


Figure 7: Weekday vs Weekend Comparison for each meter type

tamps to represent daily and seasonal variations. To model historical dependencies, lag features (lag_24h, lag_72h, and lag_168h) were created, capturing consumption from the previous day, three days prior, and one week prior. These features reflect the strong temporal continuity in energy usage. Additionally, rolling statistics were computed to capture local trends, including 24-hour and 7-day moving averages and standard deviations. A forward shift was applied to prevent data leakage, ensuring that only past information was used. Together, these features enable the model to learn both short-term fluctuations and longer-term consumption patterns.

The distribution of meter readings was highly right-skewed, with most values concentrated at lower levels and a small number of extreme values, particularly for steam meters. To address this, a log transformation (log1p) was applied to stabilise variance and improve model performance, aligning with the RMSLE evaluation metric. Further investigation identified a large number of zero readings. While many zeros represent valid periods of low or no energy consumption, anomalies were detected in Site 0 electricity data between January and May 2016, where approximately 93.75% of readings were zero, including long consecutive sequences. This indicated sensor or data collection issues. As a result, all Site 0 electricity data prior to 20 May 2016 was removed. Additionally, building-meter combinations exhibiting continuous zero readings for more than 168 hours were excluded, as these were considered unreliable. A time-based train-validation split was applied, with data from January to October 2016 used for training and November to December reserved for validation. This approach prevents data leakage and ensures realistic model evaluation, particularly when using lag-based features.

3.2 Model Choices & Justification

Three gradient-boosted tree models and one bagging ensemble were selected: LightGBM, XGBoost, CatBoost, and Random Forest. A Linear Regression baseline was included as well for comparison.

The choice of gradient-boosted trees is directly validated by the ASHRAE Great Energy Predictor III competition itself. Miller et al. [1] reported that the most accurate workflows used large ensembles of gradient-boosting models, with LightGBM dominating many top-performing solutions. The BUDS Lab summary of the competition further showed that all top-5 solutions incorporated LightGBM and/or CatBoost as primary models [2].

LightGBM was selected because of its leaf-wise tree growth strategy, histogram-based splitting, and computational efficiency when handling very large datasets. Miller, Liu, and Fu also demonstrated that LightGBM-based pipelines combined with careful preprocessing and feature engineering achieved state-of-the-art performance in the competition [6]. CatBoost was additionally selected because of its ordered boosting strategy and strong handling of categorical variables, which is particularly useful for the 16 one-hot encoded primary-use categories. This choice is supported by both the IEEE Bayesian optimisation study [9] and the CatBoost meta-heuristic forecasting work by Qu and Liu [10].

XGBoost uses depth-wise tree growth, producing prediction patterns that differ slightly from LightGBM and therefore providing useful ensemble diversity. Comparative studies of boosting methods

and XGBoost quantile regression approaches showed that XGBoost performs strongly for nonlinear building-energy forecasting tasks while also supporting interpretability through feature-importance analysis [7, 8]. Random Forest was included as a lower-correlation bagging-based alternative, where independently trained trees help reduce ensemble variance compared to purely boosted approaches. “

3.3 Loss Function & Optimisation Target

All models were trained on the log1p-transformed target (`log_meter_reading`) to directly align training with the RMSLE evaluation metric. CatBoost used an explicit `loss_function='RMSE'` on the log-scale target. LightGBM and XGBoost used the default squared-error loss on the same transformed target. Random Forest minimises mean squared error natively. Predictions are converted back to the original scale via `np.expm1()` before evaluation.

3.4 Evaluation Metric

Submissions are scored on **Root Mean Squared Log Error (RMSLE)**:

$$\text{RMSLE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(\hat{y}_i + 1) - \log(y_i + 1))^2}$$

RMSLE penalises under-predictions more than over-predictions on a multiplicative scale, making it appropriate for meter readings that span several orders of magnitude. A custom `rmsle()` function is used consistently across all notebooks for fair model comparison.

3.5 Anomaly Removal

After getting poor accuracy from initial training, three anomaly criteria were applied to the merged data. Zero-reading streaks longer than 168 hours were removed to exclude sustained flat-zero periods for each building-meter pair, which typically indicated sensor failure or installation periods. Statistical outliers, defined as readings above the 99.9th percentile, were flagged and removed. Sudden jumps, where readings exceeded ten times the previous hour’s reading (with the previous reading non-zero), were treated as meter resets and removed. Additionally, electricity data from Site 0 before 2016-05-20 was excluded due to a confirmed unit inconsistency between kBTU and kWh.

4 Results

Five model families were compared on the shared validation data: LightGBM, XGBoost, CatBoost, Random Forest, and Linear Regression. The strongest single-model result in the project comes from the tuned CatBoost model, which achieved an overall RMSLE of 0.5022. This outperformed LightGBM at 0.5084, XGBoost at 0.5210, Random Forest (depth=20) at 0.5266, and Linear Regression at 1.5989.

Model	Electricity	Chilled Water	Steam	Hot Water	Overall
CatBoost	0.2433	0.6576	0.7098	1.0780	0.5022
LightGBM	0.2436	0.6623	0.7298	1.0857	0.5084
XGBoost	0.2612	0.6896	0.7297	1.0927	0.5210
Random Forest (depth=20)	0.2599	0.7090	0.7315	1.1012	0.5266
Linear Regression	0.9970	2.1877	2.3859	2.1438	1.5989

Table 1: Per-Meter RMSLE Comparison

Performance also varied by meter type. For Electricity, tuned CatBoost was best with 0.2433, followed very closely by LightGBM at 0.2436; XGBoost and Random Forest were higher at 0.2612 and 0.2599 respectively. For Chilled Water, CatBoost again led at 0.6576, with LightGBM at 0.6623, XGBoost at 0.6896, and Random Forest at 0.7090. For Steam, CatBoost achieved the lowest RMSLE at 0.7098, ahead of XGBoost at 0.7297, LightGBM at 0.7298, and Random Forest at 0.7315. For Hot Water, the order was the same: CatBoost 1.0780, LightGBM 1.0857, XGBoost 1.0927, and Random Forest 1.1012.

Linear Regression performed much worse across every meter category, especially for Chilled Water and Steam, where RMSLE values were above 2.0.

Overall, the results show that ensemble tree models are the most reliable approach for this dataset, with tuned CatBoost as the strongest single model and LightGBM / XGBoost as close, stable alternatives.

4.1 Ensemble Methods

To further improve predictive performance, ensemble methods were explored using both soft and hard voting approaches. Soft voting assigns weights to models based on their performance, allowing stronger models such as CatBoost and LightGBM to contribute more to the final prediction. The results show that the best soft voting configuration, combining CatBoost and LightGBM, achieved the lowest RMSLE of 0.4997, outperforming all individual models. In contrast, including additional models such as Random Forest and Linear Regression reduced performance, demonstrating that weaker models can negatively impact ensemble results. Compared to hard voting, soft voting consistently achieved better performance, highlighting the importance of weighted model combination in improving predictive accuracy.

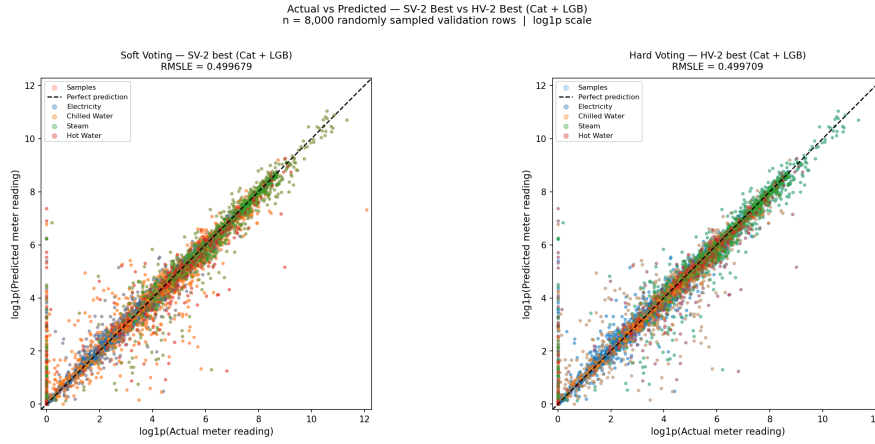


Figure 8: Hard Voting Vs Soft Voting

5 Discussion

Across LightGBM, XGBoost, CatBoost and Random Forest, error was consistently lowest for electricity (meter 0) with per-meter RMSLE near 0.24–0.26, followed by chilled water (meter 1) and steam (meter 2) in the 0.65–0.74 range, while hot water (meter 3) was the hardest at roughly 1.07–1.11. The same ranking held across all tree models, indicating the difficulty is data-driven rather than model-specific.

5.1 Comparison to Baseline Linear Regression

Linear Regression achieved a validation RMSLE of 1.5989 and the median baseline 1.3254, whereas the best individual model (CatBoost) reached 0.5022 and the ensemble methods improved further. This represents a 68% reduction in RMSLE over linear regression, demonstrating that the meter, weather and calendar relationship is strongly non-linear and interaction-heavy, which gradient-boosted trees capture but a linear model cannot.

5.2 Possible Reasons Why Electricity Was More Accurate Than Hot Water

Electricity is metered in every building and follows highly regular daily and weekly occupancy cycles, giving the boosted trees strong, repeatable signals from hour, is_business_hours and the lag features.

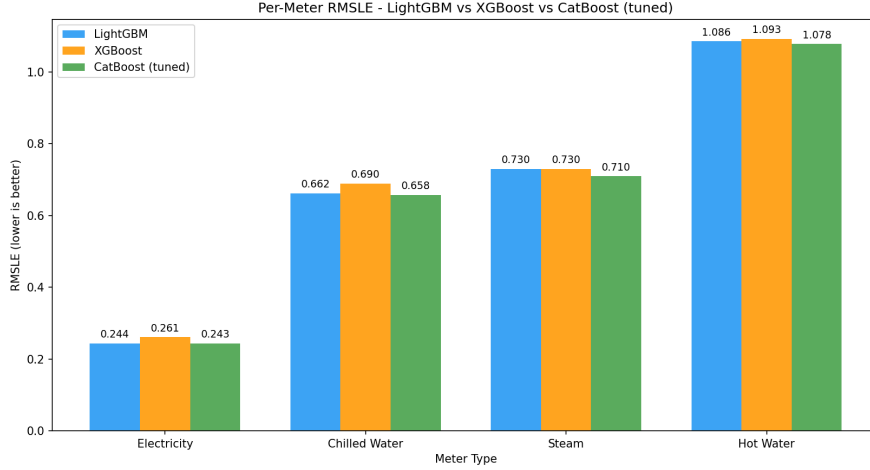


Figure 9: Per Meter RMSLE of the Boosting Models

Hot water, in contrast, is installed in far fewer buildings, is dominated by intermittent and weather-driven demand, contains long zero-streaks, and shows abrupt seasonal switching, all of which weaken temporal regularity and inflate log-scale errors.

5.3 Impact of Site 0 Anomaly Removal

Removing Site 0 electricity readings before 2016-05-20 (kBTU vs kWh unit inconsistency) along with zero-streak and 99.9th-percentile outliers produced the single largest validation gain. Before cleaning, Site 0 alone contributed disproportionately to squared log error; afterwards, validation RMSLE improved across all models and the per-site error distribution became markedly more uniform, confirming the anomaly was a label-quality issue rather than a modelling failure.

5.4 Where Lag Features Helped Most

The lag_24h, lag_168h and rolling_mean_24h features ranked among the top-five LightGBM and XGBoost importances[10]. Their benefit was greatest for electricity and chilled water, where consumption is strongly auto-correlated on daily and weekly cycles. Gains were smaller for steam and hot water, whose readings are sparser and more weather-shock driven, so recent history is a weaker predictor of the next hour.

5.5 Hyperparameter Tuning

Hyperparameter tuning using RandomizedSearchCV further reduced the RMSLE error of CatBoost, LightGBM, XGBoost, and Random Forest models. Different parameter combinations were explored for each model, including the number of estimators, learning rate, tree depth, regularization parameters, and leaf-related constraints. Due to the large size of the training dataset, random subsets of the training data were used during the search process to reduce computational cost and training time. Cross-validation was applied during tuning to improve model generalization and reduce overfitting. For CatBoost and LightGBM, a larger hyperparameter search space was explored because these models showed the strongest baseline performance. The tuned models were then retrained on the full training dataset using the best parameter combinations obtained from the search process. Overall, hyperparameter optimization provided moderate performance improvements and helped identify more stable model configurations for the final evaluation.

5.6 Ensemble Weighting

Both Hard Voting and Soft Voting were evaluated using different variations of LightGBM, XGBoost, CatBoost, Random Forest and Linear Regression. Out of all of them, Soft Voting using the best two models, CatBoost and LightGBM achieved the best ensemble RMSLE[11].

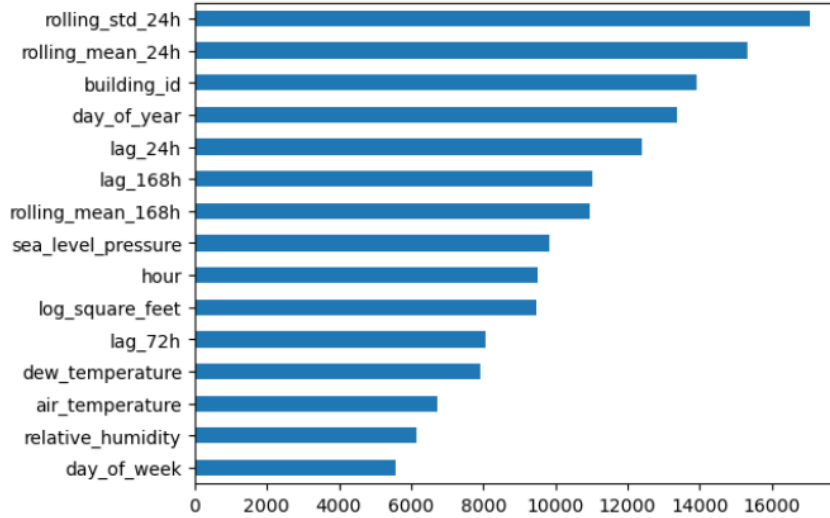


Figure 10: Top 15 Feature Importance

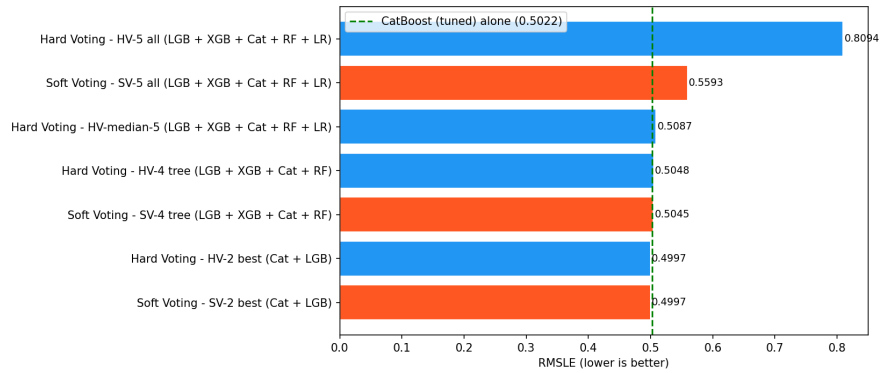


Figure 11: Hard Voting Vs Soft Voting

5.7 Error Analysis

Residual inspection showed errors are not uniformly distributed: the largest log-residuals concentrate on (i) very low or zero readings that the model slightly over-predicts, (ii) extreme summer-peak steam and hot-water spikes that are under-predicted, and (iii) buildings with short training history. RMSLE also penalises under-prediction more heavily than over-prediction, which amplifies the visible error on peak hot-water hours. Errors were near-zero-mean per building but heavy-tailed, consistent with sensor noise and unmodelled occupancy events.

6 Limitations

Despite showing some promising results, the project has several limitations. First, the top models reached an RMSLE of approximately 0.4997. This shows potential, but it is still a mediocre result. Second, the large size of the dataset created significant computational challenges. The final training dataset had around 20,216,100 records and 49 features, including engineered time-related and weather-related features. Training on the complete dataset needed a lot of computational resources. Only two of the six team members could train the entire dataset on their personal computers. Even Google Colab Enterprise could not train models on the final processed dataset. Because of these computational and time limitations, it was not possible to fully test all preprocessing methods, feature engineering strategies, and hyperparameter combinations. Additionally, although ensemble-based methods were explored, they did not lead to significant improvements over the best performance previously achieved.

7 Future Work

To further enhance the performance and robustness of the system, several improvements can be explored in the future. More advanced feature engineering techniques, particularly building-specific temporal features and seasonal trend analysis, could improve the model’s ability to capture long-term consumption behavior. Extensive hyperparameter optimization using automated search methods such as Grid Search, Random Search, or Bayesian Optimization may also lead to better predictive performance. Additionally, transformer-based or recurrent deep learning architectures designed for time-series forecasting can be explored. Using more powerful computational resources would allow for more extensive experimentation. Finally, incorporating external contextual information such as holidays, occupancy patterns, or regional energy usage trends may help improve prediction accuracy and model generalization.

8 Conclusion

Overall, the project shows that reliable energy prediction depends as much on careful preparation as on model choice. Joining the meter, building and weather data created a complete modelling table, while the main cleaning decisions removed known calibration and outage artefacts before feature engineering. Handling metadata and weather gaps consistently also made the inputs more comparable across buildings, sites and time. The strongest results came from tree-based methods, with a small additional gain from a weighted ensemble. This supports the use of flexible non-linear learners for hourly building-energy data, but it also shows that well-structured features and chronological validation are central to fair evaluation. Future work should test robustness over later periods, compare errors by site and meter type, and explore stacking only after data quality and leakage controls are secured.

References

- [1] C. Miller *et al.*, “The ashrae great energy predictor iii competition: Overview and results,” *Science and Technology for the Built Environment*, vol. 26, no. 10, pp. 1427–1447, 2020.
- [2] B. Lab, “Ashrae great energy predictor iii - top 5 winning solutions explained,” GitHub.
- [3] M. Motoki and I. Yamashita, “1st place solution — team isamu & matt,” Kaggle ASHRAE Great Energy Predictor III Writeups, Jan 2020.
- [4] cHa0s Team, “2nd place solution,” Kaggle ASHRAE Great Energy Predictor III Discussion, 2020.
- [5] eagle4, “3rd place solution,” Kaggle ASHRAE Great Energy Predictor III Writeups, 2020.
- [6] C. Miller, H. Liu, and C. Fu, “Gradient boosting machines and careful pre-processing work best,” *arXiv preprint arXiv:2202.02898*, 2022.
- [7] “Building energy consumption forecasting: A comparison of gradient boosting models,” in *Proceedings of the International Conference on Advances in Information Technology*.
- [8] “Interpretable building energy performance prediction using xgboost quantile regression,” *ScienceDirect*.
- [9] “Building energy consumption prediction using catboost with hybrid random search and bayesian optimization,” *IEEE Xplore*.
- [10] X. Qu and Z. Liu, “Forecasting total building energy using catboost and meta-heuristic algorithms,” 2026.